Règles métier

Nous décrivons ici les règles métier qui viennent compléter celles qui sont induites par le schéma de la base de données.

Préambule

> **Warning**
>
> Comme déjà évoqué précédemment, **vous devez définir vos *endpoints* en ne donnant accès qu'à des fonctions PostgreSQL**. Cela vous permettra de garder le contrôle sur les données exposées et les actions autorisées.
>
> On distingue deux types de règles métier :
>
> 1. **Règles invariantes** : Ces règles sont permanentes et ne dépendent pas des droits d'accès.
>
>   Elles doivent être gérées directement dans la base de données grâce aux outils de la base de données comme l'**intégrité référentielle**, les **contraintes d'unicité** ou de ***check***, ou encore les ***triggers***.
>
>   L'idée est de garantir que les données respectent en permanence ces règles métier, indépendamment de la manière dont elles sont mises à jour (via une requête SQL, l'interface graphique du plugin Database d'IntelliJ, pgAdmin, Postman, l'application, ...).
>
>   Par exemple, si on exécute une requête SQL `UPDATE` pour modifier la capacité d'une table utilisée dans une réservation confirmée, un trigger doit être déclenché pour vérifier que le nombre de convives de la réservation ne dépasse pas la capacité totale des tables associées compte tenu de la nouvelle capacité, et refuser la modification si c'est le cas.
> 2. **Règles dépendant des droits d'accès** : Ces règles varient en fonction des privilèges des utilisateurs.
>
>   Elles doivent être implémentées **dans les fonctions des *endpoints***.
>
>   Elles seront décrites dans la seconde partie du cahier des charges du projet.

Remarques

- Les contraintes d'unicité doivent être vérifiées indépendamment de la casse (`title` est équivalent à `Title`) et après *trim* des valeurs.
- Les longueurs maximales des champs ne sont pas systématiquements spécifiées ici. Lorsqu'elles sont omises, cela signifie qu'elles sont déterminées par la définition des colonnes dans la base de données.
- Toutes les vérifications de longueurs doivent être faites après *trim* des valeurs.
- Les clés primaires simples ou composées ne peuvent pas être modifiées (`UPDATE`).
- Certaines règles métier sont numérotées (BR-*xx*) pour faciliter leur référencement. Ce sont généralement des règles qui nécessitent le développement d'un ou de plusieurs triggers. Elles doivent être implémentées dans des fichiers SQL séparés.

Règles invariantes faisant intervenir un seul record

Utilisateurs

- Le nom complet doit avoir une longueur d'au moins 3 caractères.
- Le mot de passe doit avoir au minimum une longueur de 8 caractères, doit contenir au moins un chiffre, une lettre majuscule, une lettre minuscule et un caractère spécial parmi les suivants : `,;.:!?/$%&@#` .
- Le mot de passe doit être stocké de manière sécurisée (hachage + sel).
- On ne peut pas modifier le rôle d'un utilisateur.

Restaurants

- Le nom doit avoir au minimum une longueur de 5 caractères.
- L'adresse doit avoir au minimum une longueur de 5 caractères.
- La ville doit avoir au minimum une longueur de 3 caractères.
- Le numéro de téléphone doit avoir un format de numéro belge valide, et peut être encodé avec ou sans espaces de séparation entre les groupes de chiffres.
- La description, si elle est fournie, doit avoir au minimum une longueur de 10 caractères.
- La note doit être comprise entre 0.0 et 5.0 si elle est définie.
- La fourchette de prix doit être comprise entre 1 et 4 si elle est définie.
- La durée de créneau de réservation doit être soit 10, 15, 20, 30 ou 60 minutes.

Réservations

- Le statut d'une réservation doit être soit `pending` (en attente de confirmation), `confirmed` (confirmée), `cancelled` (annulée) ou `completed` (terminée suite à la venue effective du client).
- Le nombre de convives d'une réservation doit être strictement positif.
- Le texte des demandes spéciales d'une réservation (`special_requests`), s'il est défini, doit avoir une longueur minimale de 10 caractères.
- (BR-11) Une réservation terminée ou annulée ne peut pas être modifiée.
- On ne peut pas modifier le restaurant ou le client d'une réservation.
- (BR-09) Une nouvelle réservation doit avoir le status `pending` . Une réservation existante ne peut pas revenir "en arrière" dans son cycle de vie. Les transitions autorisées sont :
  - `pending -> confirmed`
  - `pending -> cancelled`
  - `confirmed -> completed`
  - `confirmed -> cancelled`

  La transition `confirmed -> pending` est autorisée dans certains cas (voir règles temporelles).

  Aucune transition depuis `completed` ou `cancelled` n'est autorisée.

Tables

- Le numéro de table doit être strictement positif et unique au sein d'un même restaurant.
- La capacité doit être strictement positive.
- On ne peut pas modifier le restaurant d'une table.

Services

- Le jour de la semaine doit être compris entre 1 (lundi) et 7 (dimanche).
- L'heure de fin doit être postérieure à l'heure de début d'au moins une heure. Note : l'heure de fin d'un service représente l'heure au-delà de laquelle le restaurant n'accepte plus de clients.
- On ne peut pas modifier le restaurant d'un service.

Règles invariantes faisant intervenir plusieurs records/tables

- (BR-01) Une réservation doit être associée à un utilisateur ayant le rôle de client.

- (BR-02) Le nombre de convives d'une réservation confirmée ou terminée ne peut pas dépasser la capacité totale des tables associées à la réservation. Ceci implique qu'une réservation confirmée ou terminée doit avoir au moins une table réservée appartenant au restaurant de la réservation.
- (BR-03) Les tables associées à une réservation doivent appartenir au restaurant de la réservation.
- (BR-04) L'heure d'une réservation confirmée, terminée ou en attente doit être comprise entre l'heure de début et l'heure de fin d'un service pour le même jour de la semaine.
- (BR-05) Au moment de sa création ou de sa modification, l'heure d'une réservation doit correspondre à un créneau de réservation (*slot*) pour le restaurant. Un créneau de réservation doit toujours correspondre à une heure exacte (sans minutes) augmentée d'un multiple de la durée du créneau de réservation.
- (BR-06) Une table ne peut pas faire l'objet de deux réservations terminées ou confirmées pour une même date et un même service.
- (BR-07) Pour un même jour de la semaine, les services d'un restaurant ne peuvent pas se chevaucher.
- (BR-12) Une réservation en attente ou annulée ne peut être associée à aucune une table.
- (BR-08) Au moment de leur création ou de leur modification, les heures de début et de fin d'un service doivent être alignées sur les créneaux de réservation pour le restaurant.
- (BR-10) Un restaurant doit avoir au moins un manager.
- (BR-13) Un client ne peut pas avoir deux réservations en attente, confirmées ou terminées pour une même date et un même service.

Règles temporelles

Pour ces règles, on prend en compte le temps simulé s'il est défini, sinon on prend le temps réel.

- Le changement de temps simulé n'a pas d'effet sur le statut des réservations existantes.
- (BR-14) Une réservation ne peut pas être créée dans le passé (ne doit être vérifié qu'au moment de la création).
- En plus des transitions décrites ci-dessus, le statut d'une réservation peut passer de `confirmed` à `pending` uniquement si la réservation est dans le futur (pour permettre l'édition par le client).

> Remarque : Les changements temporels qui reviennent dans le passé sont autorisés, même s'ils peuvent mener à des incohérences. Par exemple une réservation terminée dans le

> passé (ce qui est normal) restera dans ce statut même si le changement temporel l'amène dans le futur.

Règles d'accès

Les règles d'accès seront décrites dans la seconde partie du cahier des charges du projet et devront être implémentées dans les fonctions des *endpoints*.

> ⚠ **Cas particulier de l'administrateur**
>
> Le rôle d'administrateur n'est pas implémenté. Il est cependant couvert automatiquement par le fait que l'utilisateur `postgres` a tous les droits d'accès sur les tables de la base de données.
>
> Cependant, une fois la base de données créée et les règles métier mises en place, même l'utilisateur `postgres` doit s'y soumettre.
>
> Prenons un exemple pour illustrer cela : l'administrateur peut supprimer une table d'un restaurant, ce que ne peut pas faire un manager. Cependant, une telle suppression implique de supprimer par cascade les assignations de cette table à des réservations, ce qui ne doit être autorisé que si la règle métier sur la capacité totale des tables associées à ces réservations n'est pas violée.
>
> Vous devez aussi vérifier que votre implémentation des règles métier ne perturbe pas les droits d'accès de l'administrateur. Par exemple, l'administrateur doit pouvoir supprimer un restaurant ou un utilisateur.